



EcoEvoBio2026 - Análises

Autores:

- Fabio Freitas¹
- Júlia Braga¹
- Júlia Ferreira¹
- Renato Santos^{1, 2}

¹Laboratório de Bioinformática e Multi-ômicas de Microrganismos (LBMM), Instituto de Biociências, UNESP - Rio Claro

²Laboratório de Ecologia e Sistemática de Fungos (LESF), Instituto de Biociências, UNESP - Rio Claro

O presente trabalho pretende compreender as diferenças funcionais, principalmente relacionado a produção de antifúngicos identificados por BGCs, entre os microrganismos comuns e da Matéria Escura Microbiana associadas às formigas cultivadoras de fungos.

As amostras de metagenomas serão montados a partir dos dados provenientes do trabalho de Khadempur et al. (2020).

Dados e organização:

https://drive.google.com/drive/folders/1786Nt8HNwvIJUA_3tekrmbYEq3BX3ou?usp=sharing

1. Pré-processamento

As sequências Interleaved fastq foram separadas em arquivos R1.fastq e R2.fastq

```
#!/bin/bash
#PBS -N deinterleave_reads
#PBS -q parl6
#PBS -l nodes=1:ppn=16
#PBS -e deinterleave_reads_erros.txt
#PBS -o deinterleave_reads_saida.txt

# Volta ao diretório de submissão
cd "$PBS_O_WORKDIR"

for SAMPLE in *.gz;
do
    echo $SAMPLE;
    gunzip -c $SAMPLE | paste - - - - - | tee >(cut -f 1-4 | tr "\t" "\n" >
"${SAMPLE%.fastq.gz}_R1.fastq" ) | cut -f 5-8 | tr "\t" "\n" > "${SAMPLE%.fastq.gz}_R2.fastq"
```

```
q.gz}_R2.fastq";
gzip "${SAMPLE%.fastq.gz}_R1.fastq";
gzip "${SAMPLE%.fastq.gz}_R2.fastq";
done
```

1.1 Fastqc e Multiqc

```
#!/bin/bash
#PBS -N QC_reads
#PBS -q par16
#PBS -l nodes=1:ppn=16
#PBS -e QC_reads_erros.txt
#PBS -o QC_reads_saida.txt

# Volta ao diretório de submissão
cd "$PBS_O_WORKDIR"

module load anaconda3/2025.06-0
conda activate qualcheckseq

mkdir QC/;

fastqc *R*.fastq.gz -o QC/ -t 16;

multiqc QC/
```

1.2 Fastp v0.23.4

Foram mantidas as *reads* maiores 60pb e com qualidade maior que 20.

```
#!/bin/bash
#PBS -N trim_reads
#PBS -q par16
#PBS -l nodes=1:ppn=16
#PBS -e trim_reads_erros.txt
#PBS -o trim_reads_saida.txt

# Volta ao diretório de submissão
cd "$PBS_O_WORKDIR"

module load anaconda3/2025.06-0

conda activate cleanfastq
```

```

for FASTQ1 in $(ls ../RawData/*R1.fastq.gz);
do
FASTQ2=${FASTQ1/R1.fastq.gz/R2.fastq.gz}
OUTFASTQ1=`echo $FASTQ1 | cut -f3 -d '/' | sed 's/fastq.gz/filtered.fastq.gz/
g'`
OUTFASTQ2=`echo $FASTQ2 | cut -f3 -d '/' | sed 's/fastq.gz/filtered.fastq.gz/
g'`
fastp -i $FASTQ1 -I $FASTQ2 -l 60 -q 20 --thread 16 -o $OUTFASTQ1 -O $OUTFAST
Q2;
done;

deactivate cleanfastq
conda activate qualcheckseq

mkdir QC/;

```

O fastqc e o multiqc foram utilizados novamente, conforme o passo anterior para verificar a qualidade e quantidade dos dados filtrados.

Conforme observado nas análises realizadas com o FastQC e o MultiQC antes da etapa de filtragem, os dados de sequenciamento apresentaram alta qualidade, com a maior parte das sequências exibindo escore de qualidade médio superior a Q30. Assim, mesmo com uma filtragem utilizando um limiar de Q20, muitas sequências foram mantidas. Como demonstrado na tabela a seguir, na maioria das amostras mais de 97% das sequências foram mantidas. Além disso, as sequências que possuíam grandes repetições de "N" foram removidas, melhorando a qualidade dos dados para a montagem.

https://docs.google.com/spreadsheets/d/17mj_YFDNWO7wTZTNyRr5SuEvUkgdtkGpqZvFjavQsw/edit?usp=sharing

1.3 kraken2 v2.0.8-beta (PlusPFP)

```

#!/bin/bash
#PBS -N kraken2_plusfpf
#PBS -q par128
#PBS -l nodes=1:ppn=128
#PBS -e erros_kraken2_plusfpf.txt
#PBS -o saida_kraken2_plusfpf.txt

cd $PBS_O_WORKDIR

module load anaconda3/2025.06-0
conda activate kraken2
export OMP_NUM_THREADS=128

for FASTQ in $(ls ../CleanedData/*R1.filtered.fastq.gz);
do

```

```

echo "Processing $FASTQ"
FASTQ2=`echo $FASTQ | sed 's/R1.filtered.fastq.gz/R2.filtered.fastq.gz/g'`
SAMPLE=`echo $FASTQ | cut -f3 -d '/' | sed 's/_R1.filtered.fastq.gz//g'`
mkdir $SAMPLE"_kraken2_plusfpf"
kraken2 --db ../../VinoloLab/RealData_Metagenomics_03122025/kraken2_plusfpf/db/k2_plusfpf_20260226 \
    --threads 128 --unclassified-out $SAMPLE"_kraken2_plusfpf"/"$SAMPLE"_unclassified#.fq \
    --output $SAMPLE"_kraken2_plusfpf"/"$SAMPLE"_kraken2.out --confidence 0.1 \
    --classified-out $SAMPLE"_kraken2_plusfpf"/"$SAMPLE"_classified#.fq \
    --report $SAMPLE"_kraken2_plusfpf"/"$SAMPLE"_kraken2_report.txt --gzip -compressed \
    --paired $FASTQ $FASTQ2

pigz -p 128 $SAMPLE"_kraken2_plusfpf"/"$SAMPLE"_*_classified*.fq;
pigz -p 128 $SAMPLE"_kraken2_plusfpf"/"$SAMPLE"_kraken2*;
done

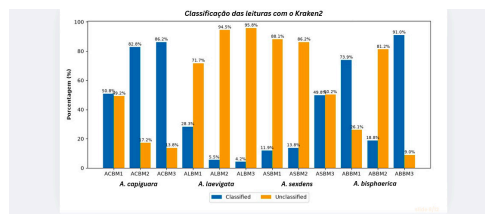
```

Haveria muita perda de dados em algumas amostras, caso as amostras fossem filtradas apenas pelo que o kraken2 classificou como bactéria. Por exemplo *Atta laevigata* ALBM2, *Atta laevigata* ALBM3, *Atta sexdens* ASBM1, *Atta sexdens* ASBM2 e *Atta Bisphaerica* ABBM2. Isto ocorreu porque muitas leituras não foram classificadas usando o kraken2 com a PlusPFP como observado na Figura a seguir, podendo ser contaminação ou sequências da matéria escura.

message.txt

Gráfico do Kraken2

kraken2_perc_classificado.py



1.4 MinHash v2.1

Tutorial utilizado como referência (<https://github.com/khidalgo85/Binning>)

O MinHash foi utilizado para calcular as distâncias das amostras (datasets), e assim, analisar se é realizar co-assembly.

```

#!/bin/bash
#PBS -N calc_distances_reads
#PBS -q par128
#PBS -l nodes=1:ppn=128
#PBS -e calc_distances_erros.txt
#PBS -o calc_distances_saida.txt

module load anaconda3/2025.06-0
conda activate minhash

# Volta ao diretório de submissão
cd "$PBS_O_WORKDIR"

for FASTQ in $(ls ../CleanedData/*R1.filtered.fastq.gz);
do
    NAME="${FASTQ%_R1.filtered.fastq.gz}"
    SAMPLE=`echo $NAME | cut -f3 -d '/'`
    zcat "$FASTQ" "${NAME}_R2.filtered.fastq.gz" > "${SAMPLE}.fastq"
    pigz -p 128 "${SAMPLE}.fastq"
done

mash sketch -o reference.msh *.fastq.gz -p 128

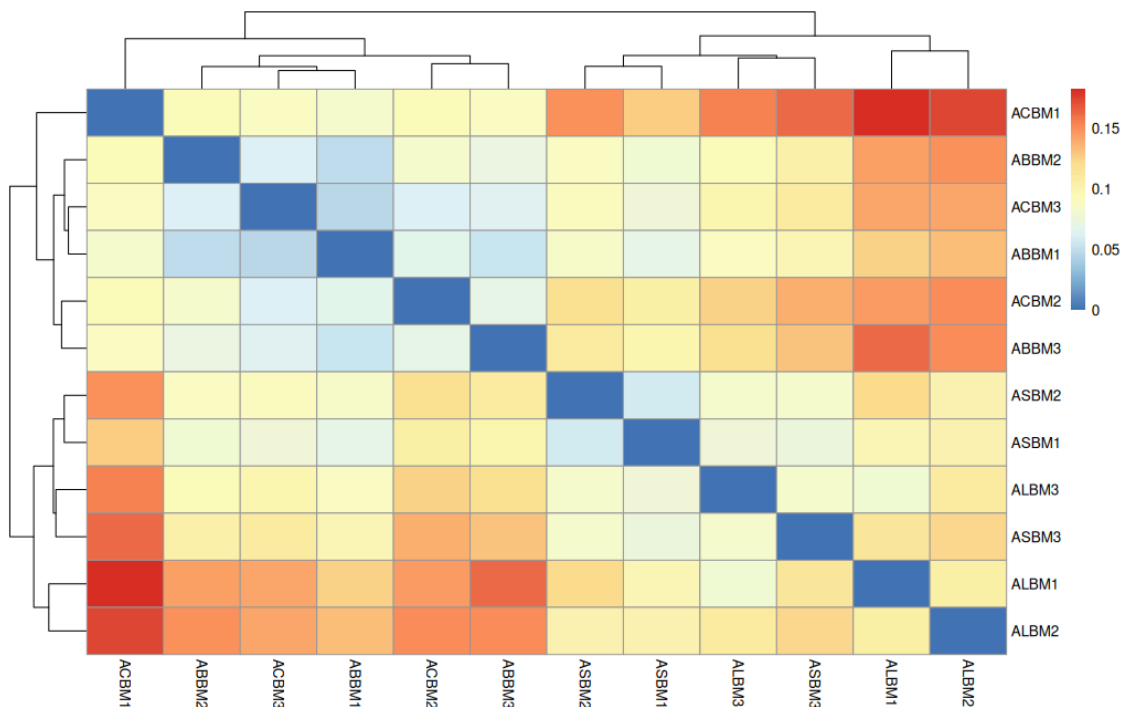
mash info reference.msh

mash dist -p 128 reference.msh reference.msh -t > distances.tsv

```

Heatmap com os agrupamentos obtidos

MinHash_heatmap.R



Os resultados demonstraram um agrupamento das amostras consistente com o observado pelo trabalho da Khadempur, onde os agrupamentos são fortemente afetados pelo tipo de alimentação da formiga. As amostras de formigas em que a base alimentar do fungo são folhas de dicotiledôneas aparentam estar mais próximas entre si, ao passo as amostras dos fungos que têm como base alimentar as gramíneas também parecem estar mais próximas.

2. Assembly

2.1 MEGAHIT v1.2.9

Assembly individual - fastq filtrados (saída do fastp)

```
#!/bin/bash
#PBS -N assembly_clean
#PBS -q par128
#PBS -l nodes=1:ppn=128
#PBS -e assembly_cleaned_reads.txt
#PBS -o assembly_cleaned_reads.txt

# Volta ao diretório de submissão
cd "$PBS_O_WORKDIR"

module load anaconda3/2025.06-0
conda activate megahit_env
export OMP_NUM_THREADS=128
```

```

for FASTQ1 in $(ls ../CleanedData/*R1.filtered.fastq.gz);
do
    FASTQ2="${FASTQ1/R1.filtered.fastq.gz/R2.filtered.fastq.gz}"
    SAMPLE=`echo $FASTQ1 | cut -f3 -d '/' | sed 's/_R1.filtered.fastq.gz//g'`

    echo "processando amostra $SAMPLE"

    megahit --k-min 21 --k-max 141 --k-step 10 \
        --memory 0.9 --num-cpu-threads 128 \
        -1 $FASTQ1 -2 $FASTQ2 -o $SAMPLE --out-prefix $SAMPLE

    pigz -p 128 $SAMPLE/${SAMPLE}.contigs.fa
done

```

3. Binning

3.1 VAMB v5.0.4

Documentação do VamB não recomenda o uso de ambientes de conda:

<https://vamb.readthedocs.io/en/latest/installation.html>

Binning do Assembly individual com fastq filtrados

```

#!/bin/bash
#PBS -N binning_clean
#PBS -q par128
#PBS -l nodes=1:ppn=128
#PBS -e erros_binning_cleaned.txt
#PBS -o saida_binning_cleaned.txt

# Volta ao diretório de submissão
cd "$PBS_O_WORKDIR"

export OMP_NUM_THREADS=128

unpigz -p 128 ..//*/*contigs.fa.gz

module load python/3.10.10-gcc-9.4.0
source vamb_env/bin/activate

/home/lovelace/proj/proj1140/rasantos/Software/vamb/src/concatenate.py all_contigs.fna.gz ..//*/*contigs.fa

```

```

deactivate

pigz -p 128 ..//*contigs.fa

module load anaconda3/2025.06-0
conda activate strobealign

mkdir aemb
for FASTQ1 in $(ls ../../CleanedData/*R1.filtered.fastq.gz);
do
    FASTQ2="${FASTQ1/R1.filtered.fastq.gz/R2.filtered.fastq.gz}"
    SAMPLE=`echo $FASTQ1 | cut -f4 -d '/' | sed 's/_R1.filtered.fastq.gz//g'`

    echo "alinhando $SAMPLE"
    strobealign -t 128 --aemb all_contigs.fna.gz "$FASTQ1" "$FASTQ2" > aemb/$SAMPLE.tsv
done

conda deactivate

module load python/3.10.10-gcc-9.4.0
source vamb_env/bin/activate

```

```

#!/bin/bash
#PBS -N binning_clean
#PBS -q par128
#PBS -l nodes=1:ppn=128
#PBS -e erros_binning_cleaned.txt
#PBS -o saida_binning_cleaned.txt

# Volta ao diretório de submissão
cd "$PBS_O_WORKDIR"

export OMP_NUM_THREADS=128

module load python/3.10.10-gcc-9.4.0
source vamb_env/bin/activate

echo "Começando o Binning"
python /home/lovelace/proj/proj1140/rasantos/Software/vamb/src/merge_aemb.py aemb abundance.tsv

vamb bin default -p 128 --outdir vamb_cleaned --fasta all_contigs.fna.gz --abundance_tsv abundance.tsv

```



```
deactivate
```

Como observado acima, o VAMB foi utilizado apenas as CPUs, porém poderia ser utilizado com GPU para otimizar o tempo.

Foram gerados 6.540 clusters na categoria fallback, 117.957 loner e 14.748 normal. O maior cluster possui 550.528.120 pb.

```
python vamb-5.0.4/src/create_fasta.py all_contigs.fna vae_clusters_split.tsv 1  
00000 out_create_fasta_100000
```

A partir do VAMB, com todas as amostras, foram criados 1513 arquivos FASTA (BINS) com mais de 100.000 pb (definido no código).

3.2 CheckM2 com todos os bins (gridUnesp)

```
#!/bin/bash  
#SBATCH --job-name=checkm2_clean_bins  
#SBATCH -t 24:00:00  
#SBATCH -c 12  
#SBATCH -e erros_checkm2_clean_bins.txt  
#SBATCH -o saida_checkm2_clean_bins.txt  
#SBATCH --mail-user=rac.santos@unesp.br --mail-type=ALL  
  
export INPUT="out_create_fasta_100000 CheckM2_database"  
export OUTPUT="checkm2_create_fasta_100000"  
export LARGE_FILES="true"  
  
module load anaconda3  
source activate checkm2_env  
  
export PATH=/home/rasantos/repositories/checkm2/bin:$PATH  
  
job-nanny checkm2 predict --input out_create_fasta_100000 --allmodels --threads 12 --database_path CheckM2_database/uniref100.K0.1.dmnd -o checkm2_create_fasta_100000
```

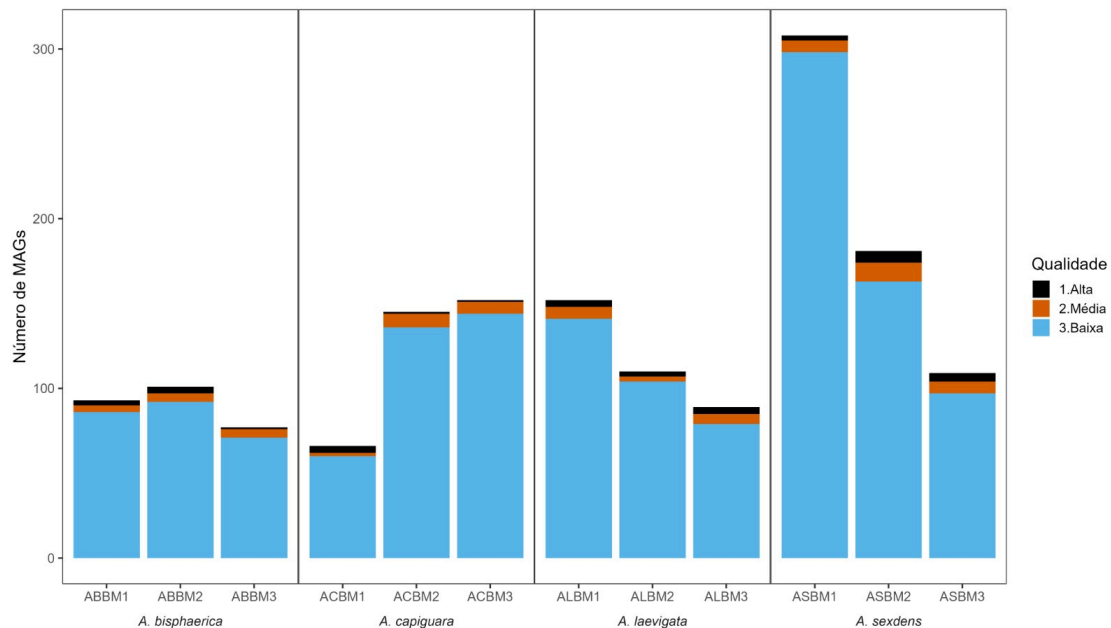
O resultado do checkM2 demonstrou que a maioria das MAGs obtidas ficaram com baixa qualidade (completude menor que 50% ou contaminação maior que 10%). Ao todo, foram geradas 71 MAGs de qualidade média (completude maior ou igual a 50% e contaminação menor ou igual que 10%) e 33 MAGs de alta qualidade (completude maior ou igual a 90% e contaminação menor ou igual que 5%).

Esses resultados podem ser reflexo de baixa profundidade ou alta complexidade das amostras, o que pode dificultar as etapas de montagem e binnig. Outros parâmetros deverão ser utilizados para

melhorar a qualidade geral das MAGs. É provável que o mais adequado seja a realização do co-assembly que utiliza informações de outras amostras para preencher possíveis lacunas presentes.

Gráfico da qualidade das MAGs obtidas

qualidade_MAGs.R



Próximas etapas

Taxonomia das MAGs

GTDB-TK v(gridUNESP)

```
#!/bin/bash
#SBATCH --job-name=gtdb-tk_clean_bins
#SBATCH -t 45:00:00
#SBATCH -c 24
#SBATCH -e erros_gtdb-tk_clean_bins.txt
#SBATCH -o saida_gtdb-tk_clean_bins.txt
#SBATCH --mail-user=fabio.rodrico@uft.edu.br --mail-type=ALL

export INPUT="VAMB_MAGs/ GTDB_database" #conferir database
export OUTPUT="taxonomy_clean_bins/ TEMP_DIR/"
export LARGE_FILES="true"

module load anaconda3
source activate gtdbtk
```

```
job-nanny gtdbtk classify_wf --genome_dir VAMB_MAGs/ --out_dir taxonomy_clean_
bins --cpus 24 --pplacer_cpus 1 --scratch_dir TEMP_DIR/ -x gz
```

Predição e anotação de BGCs

AntiSMASH v8.0.4 (erro)

```
for FASTA in out_create_fasta_100000/S*.fna;
do BIN=`echo $FASTA | cut -f2 -d '/' | sed 's/.fna//g`;
mkdir -p antismash_100000_clean_bins/$BIN;
done
```

```
#!/bin/bash
```

```
#SBATCH --job-name=antismash_clean_bins
```

```
#SBATCH -t 20:00:00
```

```
#SBATCH -c 20
```

```
#SBATCH -e erros_antismash_clean_bins.txt
```

```
#SBATCH -o saida_antismash_clean_bins.txt
```

```
#SBATCH --mail-user=fabio.rodriigo@uft.edu.br --mail-type=ALL
```

```
export INPUT="out_create_fasta_100000/S*.fna databases/"
```

```
export OUTPUT="antismash_100000_clean_bins/*"
```

```
module load miniconda/24.4.0-libmamba
```

```
source activate antismash
```

```
for FASTA in out_create_fasta_100000/S*.fna;
do BIN=`echo $FASTA | cut -f2 -d '/' | sed 's/.fna//g`;
mkdir -p antismash_100000_clean_bins/$BIN;
```

```
    job-nanny antismash -c 20 --databases databases --genefinding-tool pro
digal-m -t bacteria\
    --cc-mibig --cb-general --cb-subclusters --tfbs --asf\
    --cb-knownclusters --fullhmmmer --rre --html-ncbi-context\
    --output-dir antismash_100000_clean_bins/$BIN $FASTA;
done
```